

ArgoCD

Installation & Configuration Guide

Kubernetes GitOps Continuous Delivery

A comprehensive guide to deploying ArgoCD on Kubernetes,
configuring applications, and managing GitOps workflows.

For internal and community reference

Table of Contents

1. Introduction to ArgoCD
2. Prerequisites
3. Installation on Kubernetes
4. Initial Configuration
5. Configuring Repository Access
6. Deploying Applications (6 Service Examples)
7. Ingress & External Access
8. Pipeline Integration (CI/CD)
9. Operational Procedures
10. Troubleshooting
11. Best Practices

1. Introduction to ArgoCD

ArgoCD is a declarative, GitOps continuous delivery tool for Kubernetes. It follows the GitOps pattern of using Git repositories as the source of truth for defining the desired application state. ArgoCD automates the deployment of the desired application states in the specified target environments.

ArgoCD continuously monitors running applications and compares their live state against the desired state specified in Git. It reports deviations and can optionally auto-sync to bring the live state back in line with the desired state.

1.1 Key Concepts

- GitOps - Infrastructure and application definitions are stored in Git
- Declarative - The desired state is described, not the steps to achieve it
- Application - A group of Kubernetes resources defined by a manifest
- Sync - The process of making the live state match the desired state in Git
- Health - The status of application resources (Healthy, Degraded, Missing)
- Drift Detection - ArgoCD continuously monitors and reports configuration drift
- Self-Heal - Automatically reverts manual changes to match Git state
- Prune - Removes resources from the cluster that are no longer in Git

1.2 Architecture Components

Component	Purpose
API Server	Exposes the API, serves the Web UI, CLI, and CI/CD integrations
Repository Server	Clones Git repos and generates Kubernetes manifests
Application Controller	Monitors apps and compares live vs desired state
Dex Server	Handles SSO and external identity provider integration
Redis	Caching layer for repository and application state
ApplicationSet Controller	Automates generation of ArgoCD Applications
Notifications Controller	Sends notifications on sync/health changes

2. Prerequisites

2.1 Cluster Requirements

- Kubernetes cluster version 1.24 or later
- kubectl configured with cluster admin access
- Minimum 2 CPU cores and 4 GB RAM available for ArgoCD pods
- Ingress controller (e.g., NGINX Ingress Controller) for external access
- Container registry accessible from the cluster (for private image hosting)
- Persistent volume provisioner (optional, for Redis persistence)

2.2 Network Requirements

- Outbound HTTPS (port 443) to your Git provider (GitHub, GitLab, Azure DevOps, Bitbucket)
- Outbound HTTPS (port 443) to container registry for pulling ArgoCD images
- Internal DNS resolution for Kubernetes service discovery
- If behind a corporate firewall, ensure Git provider domains are whitelisted

2.3 Tools Required

Tool	Version	Purpose
kubectl	v1.24+	Kubernetes command-line interface
helm	v3.x (optional)	For Helm-based installation method
argocd CLI	Latest stable	ArgoCD command-line interface
git	v2.x+	Repository management
docker	v20+ (optional)	For image promotion to private registry

3. Installation on Kubernetes

3.1 Create the Namespace

Create a dedicated namespace for all ArgoCD components:

```
kubectl create namespace argocd
```

3.2 Install ArgoCD (Manifest Method)

Apply the stable installation manifest directly from the ArgoCD GitHub repository:

```
kubectl apply -n argocd -f \
  https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
```

This installs all seven ArgoCD components: API Server, Repository Server, Application Controller, Dex Server, Redis, Notifications Controller, and ApplicationSet Controller.

3.3 Install ArgoCD (Helm Method - Alternative)

For more control over configuration, use Helm:

```
helm repo add argo https://argoproj.github.io/argo-helm
helm repo update

helm install argocd argo/argo-cd \
  --namespace argocd \
  --create-namespace \
  --set server.extraArgs[0]=--insecure \
  --set server.extraArgs[1]=--rootpath=/argo
```

3.4 Verify Installation

Wait for all pods to reach Running state:

```
kubectl get pods -n argocd

# Expected output (all pods 1/1 Running):
# NAME                                READY   STATUS    RESTARTS
# argocd-application-controller-0      1/1     Running   0
# argocd-applicationset-controller-xxxxx 1/1     Running   0
# argocd-dex-server-xxxxx              1/1     Running   0
# argocd-notifications-controller-xxxxx 1/1     Running   0
# argocd-redis-xxxxx                  1/1     Running   0
# argocd-repo-server-xxxxx             1/1     Running   0
# argocd-server-xxxxx                  1/1     Running   0
```

Wait for the server deployment to become available:

```
kubectl wait --for=condition=available deployment/argocd-server \
  -n argocd --timeout=300s
```

3.5 Using a Private Container Registry

In environments where direct internet access is restricted (e.g., corporate networks), you must promote ArgoCD container images to a private registry before applying the manifest.

Step-by-step process:

1. Download the ArgoCD manifest locally

```
curl -sSL https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml \
-o argocd-install.yaml
```

2. Extract all container image references

```
IMAGES=$(grep -oP '(?<=image: ).*' argocd-install.yaml | sort -u)
echo "$IMAGES"
```

3. Pull, re-tag, and push each image to your private registry

```
PRIVATE_REGISTRY="your-registry.example.com"

for SOURCE_IMAGE in $IMAGES; do
  IMAGE_PATH=$(echo "$SOURCE_IMAGE" | cut -d '/' -f2-)
  TARGET_IMAGE="${PRIVATE_REGISTRY}/${IMAGE_PATH}"

  docker pull "$SOURCE_IMAGE"
  docker tag "$SOURCE_IMAGE" "$TARGET_IMAGE"
  docker push "$TARGET_IMAGE"

  # Rewrite the manifest to use the private registry
  sed -i "s|image: ${SOURCE_IMAGE}|image: ${TARGET_IMAGE}|g" argocd-install.yaml
done
```

4. Apply the modified manifest

```
kubectl create namespace argocd --dry-run=client -o yaml | kubectl apply -f -
kubectl apply -n argocd --server-side --force-conflicts -f argocd-install.yaml
```

4. Initial Configuration

4.1 Retrieve Initial Admin Password

After installation, retrieve the auto-generated admin password:

```
kubectl -n argocd get secret argocd-initial-admin-secret \
-o jsonpath="{.data.password}" | base64 -d && echo
```

The default username is: admin

4.2 Set a Custom Admin Password

For security, set a custom password immediately after first login:

```
# Install bcrypt if not available
pip install bcrypt

# Generate the bcrypt hash
NEW_PASSWORD="YourSecurePassword123"
BCRYPT_HASH=$(python3 -c "
import bcrypt, os
pw = os.environ['NEW_PASSWORD'].encode()
print(bcrypt.hashpw(pw, bcrypt.gensalt()).decode())
")

# Patch the ArgoCD secret
kubectl patch secret argocd-secret -n argocd --type merge \
-p '{"stringData":{"
  "admin.password":"${BCRYPT_HASH}",
  "admin.passwordMtime":"$(date -u +%FT%TZ)"
}}'

# Restart the server to pick up the change
kubectl rollout restart deployment/argocd-server -n argocd
kubectl rollout status deployment/argocd-server -n argocd --timeout=180s
```

4.3 Configure External URL and Root Path

If ArgoCD is served under a sub-path (e.g., /argo) behind an ingress controller:

```
# Set the external URL in argocd-cm ConfigMap
kubectl patch configmap argocd-cm -n argocd --type merge \
-p '{"data":{"url":"https://your-domain.com/argo"}}'

# Configure the server to use rootpath and insecure mode
kubectl patch deployment argocd-server -n argocd --type json -p='[
{
  "op": "replace",
  "path": "/spec/template/spec/containers/0/args",
  "value": [
```

```
    "/usr/local/bin/argocd-server",
    "--insecure",
    "--rootpath=/argo"
  ]
}
]'

# Wait for rollout
kubectl rollout status deployment/argocd-server -n argocd --timeout=180s
```

The `--insecure` flag disables TLS on the ArgoCD server, allowing the ingress controller to handle TLS termination. The `--rootpath` flag tells ArgoCD that it is being served at a sub-path so it can generate correct URLs for the UI and API.

5. Configuring Repository Access

ArgoCD needs read access to your Git repositories to pull Kubernetes manifests. There are multiple authentication methods available.

5.1 HTTPS with Token (repo-creds)

repo-creds define template credentials that apply to all repositories matching a URL prefix. This avoids configuring credentials for each repository individually.

```
apiVersion: v1
kind: Secret
metadata:
  name: repo-creds-https
  namespace: argocd
  labels:
    argocd.argoproj.io/secret-type: repo-creds
stringData:
  url: https://github.com/your-org
  username: git
  password: <your-personal-access-token>
  type: git
```

Apply the secret:

```
kubectl apply -f repo-creds-https.yaml
```

5.2 SSH Key Authentication

```
apiVersion: v1
kind: Secret
metadata:
  name: repo-creds-ssh
  namespace: argocd
  labels:
    argocd.argoproj.io/secret-type: repo-creds
stringData:
  url: git@github.com:your-org
  sshPrivateKey: |
    -----BEGIN OPENSSH PRIVATE KEY-----
    <your-private-key-content>
    -----END OPENSSH PRIVATE KEY-----
  type: git
```

5.3 Azure DevOps / GitHub / GitLab Integration

For Azure DevOps, use a Personal Access Token (PAT) or the pipeline System.AccessToken. For GitHub, use a fine-grained PAT. For GitLab, use a Project or Group access token.

```
# Azure DevOps example:
apiVersion: v1
```

```
kind: Secret
metadata:
  name: argocd-azure-repo-creds
  namespace: argocd
  labels:
    argocd.argoproj.io/secret-type: repo-creds
stringData:
  url: https://dev.azure.com/your-org/your-project/_git
  username: git
  password: <PAT-or-System.AccessToken>
  type: git
```

Important: If using System.AccessToken from a CI pipeline, the token expires after the pipeline run. Re-run the pipeline periodically to refresh the credential, or use a long-lived PAT instead.

6. Deploying Applications (6 Service Examples)

An ArgoCD Application is a Custom Resource Definition (CRD) that tells ArgoCD which Git repository to watch, which branch or tag to track, where to find the Kubernetes manifests within that repository, and which cluster namespace to deploy into.

6.1 Application CRD Structure

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: <application-name>           # Unique name in ArgoCD
  namespace: argocd                  # Always argocd namespace
spec:
  project: default                    # ArgoCD project (RBAC boundary)
  source:
    repoURL: <git-repo-url>          # Git repository URL
    targetRevision: <branch-or-tag>   # Branch, tag, or commit SHA
    path: <path-to-manifests>        # Directory containing K8s YAMLS
  destination:
    server: https://kubernetes.default.svc # Target cluster
    namespace: <target-namespace>       # Namespace to deploy into
  syncPolicy:
    automated:                        # Enable auto-sync (optional)
      selfHeal: <true|false>          # Auto-revert manual changes
      prune: <true|false>             # Delete orphaned resources
```

6.2 Example 1: API Gateway Service

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: api-gateway
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/your-org/api-gateway.git
    targetRevision: main
    path: k8s/production
  destination:
    server: https://kubernetes.default.svc
    namespace: app-production
  syncPolicy:
    automated:
      selfHeal: true
      prune: false
```

6.3 Example 2: User Authentication Service

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: auth-service
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/your-org/auth-service.git
    targetRevision: release/2.1
    path: deploy/overlays/prod
  destination:
    server: https://kubernetes.default.svc
    namespace: app-production
  syncPolicy:
    automated:
      selfHeal: true
      prune: true
```

6.4 Example 3: Notification Service

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: notification-service
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/your-org/notification-service.git
    targetRevision: main
    path: k8s/templates
  destination:
    server: https://kubernetes.default.svc
    namespace: app-production
  syncPolicy:
    automated:
      selfHeal: false
      prune: false
```

6.5 Example 4: Order Processing Workflow

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: order-workflow
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/your-org/order-workflow.git
    targetRevision: v3.2.0
    path: manifests/production
  destination:
    server: https://kubernetes.default.svc
    namespace: app-production
  syncPolicy:
    automated:
      selfHeal: true
      prune: false
```

6.6 Example 5: Report Generation Service

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: report-service
  namespace: argocd
spec:
  project: default
  source:
```

```
repoURL: https://github.com/your-org/report-service.git
targetRevision: main
path: .k8s/prod/templates
destination:
  server: https://kubernetes.default.svc
  namespace: app-production
syncPolicy:
  automated:
    selfHeal: false
    prune: false
```

6.7 Example 6: Frontend Web Application

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: web-frontend
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/your-org/web-frontend.git
    targetRevision: release/4.0
    path: k8s/production
  destination:
    server: https://kubernetes.default.svc
    namespace: app-production
  syncPolicy:
    automated:
      selfHeal: true
      prune: true
```

6.8 Applying Application Manifests

Apply applications individually or in bulk from a directory:

```
# Apply a single application
kubectl apply -f apps/api-gateway.yaml

# Apply all applications in a directory
kubectl apply -f apps/

# Verify applications are registered
kubectl get applications -n argocd

# Check sync and health status
kubectl get applications -n argocd \
  -o custom-columns="NAME:.metadata.name,SYNC:.status.sync.status,HEALTH:.status.health.status"
```

6.9 Sync Policy Reference

Option	Behaviour
selfHeal: true	Auto-reverts manual changes to match Git state
selfHeal: false	Drift is detected and reported but not corrected
prune: true	Resources removed from Git are deleted from cluster
prune: false	Orphaned resources are left running (manual cleanup)
No syncPolicy block	Manual sync only - no automatic deployments

7. Ingress & External Access

To access the ArgoCD UI and API externally, configure a Kubernetes Ingress resource. The examples below use NGINX Ingress Controller.

7.1 NGINX Ingress - Sub-path (/argo)

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: argocd-ingress
  namespace: argocd
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /$2
    nginx.ingress.kubernetes.io/backend-protocol: HTTP
    nginx.ingress.kubernetes.io/ssl-redirect: "true"
    nginx.ingress.kubernetes.io/proxy-body-size: "50m"
spec:
  ingressClassName: nginx
  tls:
    - hosts:
        - your-domain.example.com
      secretName: tls-secret
  rules:
    - host: your-domain.example.com
      http:
        paths:
          - path: /argo(/|$)(.*)
            pathType: ImplementationSpecific
            backend:
              service:
                name: argocd-server
                port:
                  number: 80
```

7.2 Multi-Host Ingress

For shared clusters serving multiple environments, define multiple host rules in a single Ingress resource:

```
spec:
  rules:
    - host: dev.example.com
      http:
        paths:
          - path: /argo(/|$)(.*)
            pathType: ImplementationSpecific
            backend:
              service:
                name: argocd-server
                port:
                  number: 80
```



```
- host: staging.example.com
  http:
    paths:
      - path: /argo(/|$)(.*)
        pathType: ImplementationSpecific
        backend:
          service:
            name: argocd-server
            port:
              number: 80
- host: prod.example.com
  http:
    paths:
      - path: /argo(/|$)(.*)
        pathType: ImplementationSpecific
        backend:
          service:
            name: argocd-server
            port:
              number: 80
```

7.3 Verifying Ingress

```
# Check ingress is created and has an IP
kubectl get ingress -n argocd

# Test connectivity
curl -k https://your-domain.example.com/argo
```

8. Pipeline Integration (CI/CD)

While ArgoCD handles the CD (Continuous Delivery) side, a CI pipeline is typically used to install or update ArgoCD itself, and to manage Application CRDs. Below is a high-level pipeline structure for deploying ArgoCD.

8.1 Pipeline Workflow

1. Connect to the target Kubernetes cluster (az aks get-credentials or equivalent)
2. Download the stable ArgoCD installation manifest
3. Promote container images to private registry (if network-restricted)
4. Rewrite image references in the manifest to point to private registry
5. Apply the ArgoCD manifest to the argocd namespace
6. Wait for argocd-server deployment to become available
7. Configure external URL and rootpath
8. Set the admin password (bcrypt hash)
9. Apply the Ingress resource
10. Create or update repository credential secrets
11. Apply Application CRDs from the environment-specific directory

8.2 Parameterised Pipeline (Multi-Environment)

Use a single pipeline definition with an environment parameter to deploy ArgoCD across multiple environments:

```
# Pipeline parameter definition
parameters:
  - name: environment
    displayName: "Target Environment"
    type: string
    default: dev
    values:
      - dev
      - staging
      - production

# Inline variables mapped per environment
variables:
  - ${if eq(parameters.environment, 'dev')}:
    - name: CLUSTER_NAME
      value: "dev-cluster"
    - name: REGISTRY
      value: "dev-registry.example.com"
    - name: SERVICE_CONNECTION
      value: "dev-connection"
  - ${if eq(parameters.environment, 'staging')}:
    - name: CLUSTER_NAME
      value: "staging-cluster"
    - name: REGISTRY
      value: "staging-registry.example.com"
```

```
- name: SERVICE_CONNECTION
  value: "staging-connection"
- ${{ if eq(parameters.environment, 'production') }}:
  - name: CLUSTER_NAME
    value: "prod-cluster"
  - name: REGISTRY
    value: "prod-registry.example.com"
  - name: SERVICE_CONNECTION
    value: "prod-connection"
```

8.3 GitOps Flow (After Initial Setup)

Once ArgoCD is deployed and applications are registered, the typical flow is:

1. Developer pushes code changes to a feature branch
2. CI pipeline builds, runs tests, and creates a container image
3. CI pipeline updates the image tag in the K8s manifest (in Git)
4. ArgoCD detects the manifest change in Git
5. ArgoCD syncs automatically (if automated sync is enabled) or waits for manual sync
6. ArgoCD reports the sync status and application health in the dashboard

9. Operational Procedures

9.1 Checking Application Status

```
# List all applications with sync and health status
kubectl get applications -n argocd

# Detailed YAML output for a specific app
kubectl get application api-gateway -n argocd -o yaml

# Using the ArgoCD CLI
argocd app list
argocd app get api-gateway
```

9.2 Manual Sync

```
# Using kubectl (patch approach)
kubectl patch application api-gateway -n argocd --type merge \
  -p '{"operation":{"initiatedBy":{"username":"admin"},
    "sync":{"revision":"HEAD"}}}'

# Using ArgoCD CLI
argocd app sync api-gateway

# Sync with prune (remove orphaned resources)
argocd app sync api-gateway --prune

# Via the UI: Click application -> Sync -> Synchronize
```

9.3 Hard Refresh (Clear Cache)

If ArgoCD shows stale data, force a hard refresh:

```
# CLI
argocd app get api-gateway --hard-refresh

# UI: Click application -> Refresh (hold Shift for hard refresh)
```

9.4 Rollback to Previous Revision

```
# View deployment history
argocd app history api-gateway

# Rollback to a specific revision
argocd app rollback api-gateway <history-id>
```

9.5 Viewing Logs

```
# ArgoCD Server logs (API, UI issues)
```

```
kubectl logs deployment/argocd-server -n argocd --tail=100

# Application Controller logs (sync issues)
kubectl logs statefulset/argocd-application-controller -n argocd --tail=100

# Repo Server logs (Git connectivity, manifest generation)
kubectl logs deployment/argocd-repo-server -n argocd --tail=100
```

9.6 Deleting an Application

```
# Delete app AND its cluster resources (cascade)
argocd app delete api-gateway --cascade

# Delete app but KEEP cluster resources
argocd app delete api-gateway --cascade=false

# Using kubectl
kubectl delete application api-gateway -n argocd
```

10. Troubleshooting

10.1 Common Issues and Resolutions

Symptom	Likely Cause	Resolution
Unknown sync status	Invalid repoURL or branch name	Verify repo URL and branch exist in Git
OutOfSync / Missing	Manifest path does not exist	Check the path exists in target branch
Authentication failed	Expired or invalid token/PAT	Refresh the repo-creds secret
ImagePullBackOff	Image not in private registry	Promote images to target registry
Degraded health	Pods in CrashLoopBackOff	Check pod logs and resource limits
502 Bad Gateway (UI)	argocd-server pod not running	Check pod status and restart
Sync failed - pruning	Resources have finalizers	Remove finalizers or sync without prune

10.2 Debugging Repository Connectivity

```
# Check repo-server logs for Git errors
kubectl logs deployment/argocd-repo-server -n argocd | grep -i error

# Verify the repo-creds secret exists
kubectl get secrets -n argocd \
  -l argocd.argoproj.io/secret-type=repo-creds

# Test network connectivity from inside the cluster
kubectl run test-net --rm -it --image=busybox --restart=Never -- \
  wget --spider --timeout=5 https://github.com

# Check if ArgoCD can reach the repo (via CLI)
argocd repo list
```

10.3 Debugging Sync Failures

```
# Get detailed sync status
argocd app get api-gateway --show-operation

# View the diff between live and desired state
argocd app diff api-gateway

# Check application controller logs for specific app errors
kubectl logs statefulset/argocd-application-controller -n argocd \
  | grep api-gateway
```

10.4 Resetting ArgoCD Installation

As a last resort, you can reset the ArgoCD installation:

```
# Remove and reinstall (Application CRDs will be preserved if not deleted)
kubectl delete -n argocd -f argocd-install.yaml
```

ArgoCD Installation & Configuration Guide

Kubernetes GitOps Deployment

```
kubectl apply -n argocd -f argocd-install.yaml

# Wait for recovery
kubectl wait --for=condition=available deployment/argocd-server \
  -n argocd --timeout=300s

# Re-apply applications if needed
kubectl apply -f apps/
```

11. Best Practices

11.1 Repository Organisation

- Keep Kubernetes manifests in a dedicated directory (e.g., k8s/, .k8s/, deploy/)
- Use environment-specific subdirectories (e.g., k8s/dev/, k8s/staging/, k8s/prod/)
- Store Application CRDs in the infrastructure repo, not in the application repo
- Use a separate branch or tag per environment for production releases
- Avoid storing secrets in Git - use sealed-secrets or external-secrets operator

11.2 Security

- Use short-lived tokens (e.g., pipeline System.AccessToken) where possible
- Rotate the admin password regularly and store it in a secrets manager
- Enable RBAC and create project-scoped access for individual teams
- Restrict ArgoCD to deploy only to designated namespaces using AppProjects
- Audit sync operations and maintain a log of who triggered deployments
- Use network policies to restrict ArgoCD pod communication

11.3 Sync Policies

- Start with selfHeal: false and prune: false until you build confidence
- Enable selfHeal for production-critical services to prevent configuration drift
- Enable prune only when Git is confirmed as the single source of truth
- Use sync windows to restrict when automatic syncs can occur (e.g., business hours only)
- Configure retry policies for transient failures during sync

11.4 Monitoring and Alerting

- Configure ArgoCD notifications for sync failures (Slack, Teams, email, webhook)
- Monitor ArgoCD pod resource consumption (CPU, memory) for controller and repo-server
- Set up alerts for applications stuck in Degraded, Unknown, or OutOfSync state
- Periodically review and clean up orphaned or unused applications
- Track sync frequency and duration as operational metrics

11.5 High Availability

- For production, install ArgoCD in HA mode with multiple replicas
- Use an external Redis instance or Redis HA for caching
- Distribute application controller shards across multiple pods
- Use pod anti-affinity rules to spread ArgoCD pods across nodes
- Configure resource requests and limits to prevent resource starvation
- Back up ArgoCD Application and AppProject CRDs regularly

11.6 Multi-Environment Strategy

- Use one ArgoCD instance per cluster, or a central instance managing multiple clusters
- Separate Application CRDs by environment in distinct directories (apps/dev/, apps/prod/)
- Use different targetRevision per environment (e.g., develop for dev, release/* for prod)
- Maintain consistent manifest structure across all service repositories

--- End of Document ---